

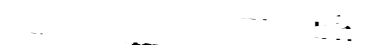
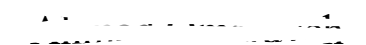


BIRZEIT UNIVERSITY

Faculty of Engineering and Technology
Computer Science Department
Comp438 – QA course

Assignment 1 Inspection Report

Students:


Mohammad Othman


Instructor: Faisal Shehadeh

❖ Introduction

This code inspection is applied to a Java Contacts Management System. The system is a console-based application that allows users to manage contacts. Users can add, search, edit, and delete contacts. The system also supports marking favorite contacts, simulating calls, and viewing call history.

The goal of this inspection is to find problems in the code by reading it carefully without running the program. The inspection focuses on common code errors, code structure and modularity, and the quality of comments and documentation. An additional goal of this inspection is to check maintainability, including code size, complexity, and duplication, to see how easy the code is to understand and maintain.

➤ **ContactDriver Class:**

This class is the main entry point of the system. It shows the menu to the user, reads user input, and controls the program flow. It connects user choices with the service layer.

➤ **Contact Class:**

This class represents a contact in the system. It stores basic contact information such as name, phone number, relation, and favorite status.

➤ **Database Class:**

This class works as an in-memory storage for the system. It stores contacts, favorites, blocked contacts, and call history using arrays.

➤ **Operations Class:**

This class contains the main business logic of the system. It handles adding, searching, editing, deleting contacts, managing favorites, and handling call history.

❖ Defect List Table

ID	Category	Location (File / Method)	Severity	Description	Suggested Fix
D1	Common code errors (exception risk)	Operations.java – add(), edit()	High	The program crashes if the user enters letters with the phone number because used without validation.	Read phone input then convert to long or use try-catch.
D2	Common code errors (input validation)	Contact.java – constructors	Medium	Contact data is not validated. Name and relation may contain invalid values, and phone numbers are not checked.	Validate input values before creating a Contact object.
D3	Modularity & structure	ContactDriver.java – help option	High	The help option calls main() again, which may lead to unnecessary recursion.	Create a separate method to print the menu instead of calling main().
D4	Common code errors (array bounds risk)	Operations.java – addUser(), callLogs()	High	If arrays are full, index remains -1.(which causes a runtime exception.)	Check if the array is full before inserting new data.
D5	Common code errors (array index risk)	Database.java – setDb(), setFavs()	High	Index values are not checked before inserting data into arrays.0 (which cause runtime exceptions.)	Validate index range before assigning value.
D6	Modularity & structure (coupling)	Database.java – public static arrays	High	Public static arrays allow direct access to data, increasing coupling between classes.	Make arrays private and access them only through controlled methods.
D7	Modularity & structure / maintainability	Operations.java – multiple methods	Medium	Multiple Scanner objects are created and input logic is repeated.	Use one shared Scanner and helper methods for user input.
D8	Common code errors (logic bug)	Operations.java – add()	Medium	String comparison uses relation != "", which may behave incorrectly.	Use relation.isEmpty() or !relation.trim().isEmpty().
D9	Logic / validation	Operations.java – edit(), editUser()	Medium	Phone editing uses nextInt() and does not check for duplicate numbers.	Use nextLong() and check for existing phone numbers.
D10	Logic / usability	ContactDriver.java – main loop	Low	After choosing “exit”, the program still asks if the user wants to continue.	Use break immediately after exit choice.
D11	Dead / incomplete functionality	ContactDriver.java – block, showblocked	Low	Menu options exist but only print “Under maintenance”, which is misleading to users.	Remove these options or complete the features.
D12	Design / maintainability	Database.java – overall design	Medium	The repository provides only set methods and no safe get or delete methods.	Add methods to read, delete, and check data safely.
D13	Separation of concerns	ContactDriver.java & Operations.java	Medium	UI logic and business logic are tightly coupled, making it hard to use	Separate UI from business logic.
D14	Documentation Issue	All classes	Low	Most public methods do not have comments.	Add short comments or JavaDoc for public methods.

❖ Code Smells & Design Issues

➤ 3.1 Cohesion and Coupling

Cohesion:

Some classes in the system have low cohesion. For example, the **ContactDriver** class is responsible for showing the menu, reading user input, and controlling the program flow, all inside one method. This makes the class harder to read and harder to modify. Even small changes in the menu or input handling may require changes in the main logic.

In addition, several methods in the **Operations** class perform multiple tasks at the same time. These methods handle user input, create contact objects, and store data in the database. Doing many things in one method reduces clarity and makes the code more difficult to understand. Splitting these tasks into smaller methods would improve readability and make the code easier to maintain.

Coupling:

The coupling between classes is high because many classes directly access data from the **Database** class. The arrays in Database are declared as public and static, which allows any class to modify them without control. This increases the chance of unintended changes and bugs.

Because of this design, any change to the database structure would affect several other classes. This makes the system harder to maintain in the long term. Reducing direct access to shared data would improve system stability.

➤ 3.2 Architecture and Design Comments

The system follows a simple layered structure that includes Driver, Service, Repository, and Entity layers. While this structure is clear, the separation between layers is weak. For example, user input handling is not limited to the Driver layer. Scanner objects are created in several methods inside the Service layer, which mixes user interface logic with business logic.

The Repository layer also has design issues. The **Database** class exposes its internal arrays directly and provides only limited methods to work with the stored data. This makes it easy for other classes to misuse the data and reduces control over how data is updated or deleted.

Another limitation of the design is the use of fixed-size arrays to store contacts, favorites, and call logs. This limits scalability and may cause runtime problems when the arrays become full. Using dynamic collections like ArrayList and hiding data access behind proper methods would make the system more flexible and easier to extend.

❖ Conclusion

This code inspection identified several issues in the Contacts Management System by carefully reviewing the source code without executing it. The inspection revealed common code errors, weak input validation, and multiple design and maintainability problems across different classes.

Some of the most important issues include the risk of runtime crashes due to unvalidated input, tight coupling between system components, and limited separation of responsibilities. In addition, the use of fixed-size arrays and publicly accessible data structures reduces the system's flexibility and increases the chance of errors.

Overall, this inspection demonstrates the value of code inspection as an effective quality assurance technique. Detecting these issues early allows developers to improve the system before deployment. By improving validation, reducing coupling, and applying better design practices, the system can become more reliable, easier to understand, and easier to maintain in the future.

❖ What We learned from This Part of the Course:

In this part of the course, we learned several important concepts related to software quality and code inspection.

First, we learned the importance of analyzing the code structure and following a layered and clean architecture. Using a clear structure makes the code easier to understand, maintain, and modify over time

Second, we learned how to search for and select a small open-source software component from a public repository such as GitHub, GitLab, or Bitbucket. This helped us work with real-world code instead of artificial examples.

Third, we learned key principles of Code Modularity and Structure, including cohesion and coupling, single responsibility principle, code duplication, long methods and classes, and improper separation of concerns. We also learned how to inspect these issues and identify them inside the source code.

Fourth, we learned the importance of proper documentation and comments in the code. Comments help developers understand the purpose of each part of the code and make future maintenance easier

Finally, in the area of maintainability metrics, we learned how to evaluate the maintainability of the code and identify factors that affect how easy the software is to understand, update, and extend.